



Universidad LaSalle Bolivia
Facultad de Ingeniería

DOCUMENTO DE DISEÑO DE PRUEBAS
Software Test Documentation — STD

Sistema de Gestión Escolar *Colegio Santa Beatriz*

Equipo de Desarrollo

 Roger

 Favio

 Fabricio

 Yhony

 **Curso:** Gestión de Proyectos de Software

 **Gestión:** 2025 – I

 **Fecha:** 23 de junio de 2026

 **Versión:** 1.0.0

Índice

1. Introducción	2
2. Estrategia y Alcance	2
2.1. Componentes probados	2
2.2. Tipos de pruebas aplicadas	2
2.3. Entorno de pruebas	3
3. Diseño de Casos de Prueba	3
3.1. Convención AAA	3
3.2. Módulo: Autenticación	3
3.3. Módulo: Gestión de Usuarios	3
4. Ejecución y Reporte de Resultados	4
4.1. Output de la suite de pruebas	4
4.2. Ejemplo de prueba unitaria (Patrón AAA)	4
4.3. Tabla de resultados	5
5. Conclusiones	5

§1 Introducción

El presente documento constituye el **Diseño de Pruebas (Software Test Documentation — STD)** del Sistema de Gestión Escolar *Colegio Santa Beatriz*, desarrollado en el marco del curso *Gestión de Proyectos de Software* de la Universidad LaSalle Bolivia.

El objetivo es demostrar que el sistema fue sometido a un proceso de verificación y validación riguroso, siguiendo estándares de la industria, y no simplemente probado de manera informal.

Alcance del documento

Este documento cubre la estrategia, diseño de casos de prueba y reporte de resultados para los módulos de **Autenticación**, **Gestión de Usuarios** y las **APIs REST** del backend desarrollado en NestJS.

§2 Estrategia y Alcance

2.1 Componentes probados

- ✔ **Backend (NestJS + Prisma):** Servicios, repositorios, controladores y lógica de negocio.
- ✔ **Autenticación (Better Auth):** Flujos de sign-in, sign-up y gestión de sesiones.
- ✔ **Base de Datos (PostgreSQL):** Integridad referencial y migraciones.
- ✗ **APIs de terceros:** Fuera del alcance (servicios de pago, email externo).

2.2 Tipos de pruebas aplicadas

Tipo	Descripción	Framework
Unitarias	Validan lógica aislada de servicios y DTOs	Jest
Integración	Verifican interacción entre módulos y DB	Jest + Supertest
E2E / Funcionales	Flujos completos de usuario via HTTP	Jest + Supertest
Regresión	Garantizan que cambios no rompen funcionalidad	CI/CD pipeline

Cuadro 1: Tipos de pruebas aplicadas al sistema

2.3 Entorno de pruebas

 **Entorno aislado:** Las pruebas se ejecutan contra una base de datos en contenedor Docker independiente (`postgres:16-alpine`) con datos sintéticos generados por seeds, garantizando que la base de datos de producción no es afectada.

§3 Diseño de Casos de Prueba

3.1 Convención AAA

Todos los casos de prueba siguen el patrón estándar de la industria **AAA**:

- **Arrange** — Preparar el escenario y los datos de entrada.
- **Act** — Ejecutar la acción bajo prueba.
- **Assert** — Verificar que el resultado coincide con el esperado.

3.2 Módulo: Autenticación

ID	Escenario	Descripción / Entrada	Resultado Esperado
TC-01	Login exitoso (Happy Path)	Usuario válido: <code>admin@santabeatriz.com / Admin123!</code>	HTTP 200 + cookie de sesión establecida
TC-02	Login contraseña incorrecta	Usuario existente con contraseña errónea	HTTP 401, mensaje de error
TC-03	Login email inexistente	Email no registrado en el sistema	HTTP 401, mensaje genérico
TC-04	Login campos vacíos	Body vacío <code>{}</code>	HTTP 422, error de validación
TC-05	Sesión caducada	Token expirado en cabecera	HTTP 401, redirección al login

Cuadro 2: Casos de prueba — Módulo Autenticación

3.3 Módulo: Gestión de Usuarios

ID	Escenario	Descripción / Entrada	Resultado Esperado
TC-06	Listar usuarios (Admin)	GET /api/users con token admin	HTTP 200, array JSON con usuarios
TC-07	Listar sin autenticación	GET /api/users sin token	HTTP 401, acceso denegado
TC-08	Email duplicado	POST con email ya registrado	HTTP 400, error de unicidad
TC-09	Crear usuario válido	POST con todos los campos correctos	HTTP 201, usuario creado con ID
TC-10	Acceso sin permisos	Rol Estudiante accede a endpoint Admin	HTTP 403, <i>Forbidden</i>

Cuadro 3: Casos de prueba — Módulo Gestión de Usuarios

§4 Ejecución y Reporte de Resultados

4.1 Output de la suite de pruebas

Resumen de Ejecución — Jest

```

PASS  src/auth/auth.service.spec.ts
PASS  src/user/user.service.spec.ts
PASS  src/app.e2e-spec.ts

Test Suites: 3 passed, 3 total
Tests:      15 passed, 0 failed, 0 skipped, 15 total
Snapshots:  0 total
Time:       8.342 s

```

4.2 Ejemplo de prueba unitaria (Patrón AAA)

```

1 describe('AuthService', () => {
2   describe('signIn', () => {
3     it('TC-01: debe retornar sesion valida con credenciales
      correctas',
4     async () => {
5       // ARRANGE: preparar datos de entrada
6       const credentials = {
7         email: 'admin@santabeatriz.com',
8         password: 'Admin123!',
9       };
10

```

```

11      // ACT: ejecutar la accion bajo prueba
12      const result = await authService.signIn(credentials);
13
14      // ASSERT: verificar el resultado esperado
15      expect(result.status).toBe(200);
16      expect(result.data.user.email).toBe(credentials.email);
17      expect(result.data.token).toBeDefined();
18    });
19  });
20 });

```

Listing 1: Prueba unitaria del AuthService — sign-in exitoso

4.3 Tabla de resultados

ID	Escenario	HTTP	Estado
TC-01	Login exitoso	200	✓ PASS
TC-02	Login contraseña incorrecta	401	✓ PASS
TC-03	Login email inexistente	401	✓ PASS
TC-04	Login campos vacíos	422	✓ PASS
TC-05	Sesión caducada	401	✓ PASS
TC-06	Listar usuarios (Admin)	200	✓ PASS
TC-07	Listar usuarios sin auth	401	✓ PASS
TC-08	Email duplicado	400	✓ PASS
TC-09	Crear usuario válido	201	✓ PASS
TC-10	Acceso sin permisos	403	✓ PASS
Total		10/10	100 %

Cuadro 4: Reporte de resultados de las pruebas

§5 Conclusiones

- El sistema superó el **100 % de los casos de prueba diseñados**, incluyendo tanto el *Happy Path* como el *Sad Path*.

- La arquitectura limpia (*Clean Architecture*) adoptada facilitó las pruebas unitarias al aislar cada capa de forma independiente.
- El entorno de pruebas aislado en Docker garantizó **reproducibilidad** e **independencia** respecto al entorno de producción.
- La ejecución automática en el pipeline CI/CD asegura regresiones continuas ante cada cambio de código.

Trabajo futuro: Se recomienda ampliar la cobertura con pruebas E2E de interfaz (Playwright/Cypress) para el frontend Next.js y pruebas de carga con *k6* o *Artillery*.